Package ‘dsClientHarmonization’

July 6, 2026

Title What the Package Does (One Line, Title Case)

Version 0.0.0.9000

Description What the package does (one paragraph).

License `use_mit_license()`, `use_gpl3_license()` or friends to pick a license

Encoding UTF-8

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.0.0

Contents

`ds.cast_NA` *Cast "NA" string into NA*

Description

Executes `cast_NADS` server-side function to convert "NA" strings into NA values across a dataset, and reports whether any replacements occurred.

Usage

```
ds.cast_NA(df, modified_obj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame to be converted.
modified_obj	A character string specifying the name of the new server-side object to store the modified data frame. If NULL, a name is generated automatically by appending "_modified" to df.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified the default set of connections will be used: see datashield.connections_d

Value

Returns the name of the newly created server-side object containing the modified data frame.

---

ds.cast_to_factor *Cast specified columns to factor on a server-side data frame*

Description

Convert one or more columns of a server-side data frame to factor using the cast_to_factorDS server-side function. The resulting data frame is assigned as a new object on each server.

Server function called: cast_to_factorDS

Usage

```
ds.cast_to_factor(df, columns, modified_obj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame containing the columns to be converted.
columns	A character vector specifying the names of the columns to convert to factor.
modified_obj	A character string specifying the name of the new server-side object to store the modified data frame. If NULL, a name is generated automatically by appending "_modified" to df.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified the default set of connections will be used: see datashield.connections_d

Value

Returns the name of the newly created server-side object containing the modified data frame.

`ds.check_missing_data` *Filter columns of a server-side object based on a missingness threshold*

Description

Computes the percentage of missing values for each variable in a distributed data set and optionally removes variables exceeding a user-defined threshold.

The function supports two filtering strategies:

- `"combined"`: computes a global missingness percentage for each variable as a weighted average across all servers. The same set of variables is removed from all servers.
- `"split"`: evaluates missingness independently on each server and removes server-specific sets of variables.

A missingness table is always displayed. When variables exceed the threshold, they are reported along with their missingness percentages.

If no variables exceed the threshold, no filtering is applied and the original data remain unchanged.

In addition to being displayed, the underlying missingness table(s) are attached as attributes to the returned value, so they can be retrieved for further use (e.g. plotting) without re-parsing the printed messages. Only the attribute relevant to the chosen `type` is populated; the other is `NULL`:

- `"missing_aggregate"`: populated only when `type = "combined"`. A single `data.frame` with variables as row names, one column per server, and a `global` column with the weighted-average missingness percentage.
- `"missing_server"`: populated only when `type = "split"`. A named list of `data.frames` (one per server), each with variables as row names and a single column containing that server's missingness percentages.

Server-side functions called: `check_missing_dataDS`, `remove_columnsDS`

Usage

```
ds.check_missing_data(
  df,
  threshold,
  type = c("combined", "split"),
  remove = c(TRUE, FALSE),
  preproc_newobj = NULL,
  datasources = NULL
)
```

Arguments

`df`	A character string specifying the name of the input data frame stored on each server.
`threshold`	A numeric value between 0 and 100 representing the maximum allowed percentage of missing values per variable.
`type`	A character string specifying the filtering strategy: `"combined"` (default) or `"split"`.

remove	A logical value indicating whether flagged variables should be removed. If TRUE (default), filtered data are stored server-side using remove_columnsDS. If FALSE, variables exceeding the threshold are only identified and returned, without modifying any server-side object.
preproc_newobj	A character string specifying the name of the output server-side object. If NULL, a name is automatically generated by appending "_preprocessed" to df. Ignored if remove = FALSE.
datasources	A list of DSConnection-class objects. If not provided, the default active connections are used.

Value

- If remove = TRUE: invisibly returns a character string with the name of the created server-side object containing the filtered data. If no variables exceed the threshold, the original data are left unchanged and the intended output object name is still returned.
- If remove = FALSE:
  - For type = "combined": a character vector of variables exceeding the threshold.
  - For type = "split": a named list of character vectors (one per server) containing variables exceeding the threshold.
  - NULL if no variables exceed the threshold.

In every case above, the returned value additionally carries two attributes, "missing_aggregate" and "missing_server", as described in the Description section, which can be retrieved via attr() (e.g. attr(result, "missing_aggregate")).

---

ds.check_patient_visit *Check patient-visit combinations on a server-side data frame*

Description

Verify all patient-visit combinations in a server-side data frame and report any duplicates using the check_patient_visitDS server-side function. Messages are printed for each server indicating whether duplicates are present.

Server function called: check_patient_visitDS

Usage

```
ds.check_patient_visit(df, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame containing at least the columns pat_ID and Visit_months_from_diagnosis.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified, the default set of connections will be used: see datashield.connections_

Value

Invisible NULL. Messages are printed to indicate whether duplicate patient-visit pairs were found on each server.

ds.check_types — *Check variable types on a server-side data frame*

Description

Verify that numeric and categorical variables in a server-side data frame comply with predefined valid ranges and values. Uses the server-side functions check_numericDS and check_categoricalDS to identify non-compliant columns. Messages are printed for each server indicating which columns, if any, are invalid.

Server functions called: check_numericDS, check_categoricalDS

Usage

```
ds.check_types(df, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame containing numeric and categorical variables to be checked.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified, the default set of connections will be used: see datashield.connections_

Value

Invisible NULL. Messages are printed to indicate which columns, if any, are non-compliant on each server.

ds.check_variables — *Check variables on a server-side data frame*

Description

Verify that a server-side data frame contains exactly the expected variables. Uses the server-side function check_variablesDS to identify missing or extra variables. Messages are printed for each server indicating discrepancies.

Server function called: check_variablesDS

Usage

```
ds.check_variables(df, variables, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame to check.
variables	A character vector specifying the expected variable names.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified, the default set of connections will be used: see datashield.connections_

Value

Invisible list of results returned by the server-side function. Each element corresponds to a server and contains two components: `missing` (variables present in `variables` but missing in the data frame) and `extra` (variables present in the data frame but not in `variables`).

`ds.fill_missing_data` *Fill missing visit data on a server-side data frame*

Description

Invoke the server-side function `fill_missing_dataDS` to fill missing values in a specified column within each patient, ordered by visit time from diagnosis. The modified data frame is assigned to a new server-side object.

Server function called: `fill_missing_dataDS`

Usage

```
ds.fill_missing_data(
  df,
  pat_id_col,
  visit_col,
  value_col,
  filled_newobj = NULL,
  datasources = NULL
)
```

Arguments

`df`	A character string specifying the name of the server-side data frame containing at least the patient identifier column, the visit time column, and the variable to be forward-filled.
`pat_id_col`	A character string specifying the name of the patient identifier column.
`visit_col`	A character string specifying the name of the visit time column.
`value_col`	A character string specifying the name of the column whose missing values should be forward-filled.
`filled_newobj`	A character string specifying the name of the new server-side object that will store the modified data frame. If not provided, the default name is `paste0(df, "_filled")`.
`datasources`	A list of `DSConnection-class` objects obtained after login. If not specified, the default set of connections is used: see `datashield.connections_default`.

Value

Invisibly returns the name of the newly created server-side object. A message is printed indicating where the result has been saved.

ds.global_imputation — *Global imputation of a server-side object using MICE*

Description

Performs imputation on data from a single source on the server-side.

Server function called: `global_imputationDS`

Usage

```
ds.global_imputation(
  df,
  id_col = NULL,
  m = 5,
  maxit = 30,
  imputed_newobj = NULL,
  datasources = NULL
)
```

Arguments

`df`	A character string specifying the name of the server-side data frame on which the global imputation is performed.
`id_col`	A character string indicating the name of the identifier column that is excluded from imputation.
`m`	A numeric value representing the number of imputations to generate. Default is 5.
`maxit`	A numeric value representing the maximum number of MICE iterations per imputation chain. Default is 30.
`imputed_newobj`	A character string specifying the name of the new imputed object to be created on each server. If NULL, a name is generated automatically by appending `"_imputed"` to `x`.
`datasources`	A list of DataSHIELD connections. If NULL, the active connections returned by `datashield.connections_find()` are used.

Value

The name of the newly created server-side object containing the imputed data set.

ds.remove_columns — *Remove specified columns from a server-side data frame*

Description

Remove one or more columns from a server-side data frame using the `remove_columnsDS` server-side function. The resulting data frame is assigned as a new object on each server.

Server function called: `remove_columnsDS`

Usage

```
ds.remove_columns(df, col_names, newobj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame from which columns will be removed.
col_names	A character vector specifying the names of the columns to remove. These columns must exist in the data frame, otherwise the server-side function will throw an error.
newobj	A character string specifying the name of the new server-side object to store the modified data frame. If NULL, a name is generated automatically by appending "_filtered" to df.
datasources	a list of DSConnection-class objects obtained after login. If the datasources argument is not specified the default set of connections will be used: see datashield.connections_d

Value

Returns the name of the newly created server-side object containing the modified data frame.

Index